

MYWORLD WHATSAPP AI PLATFORM

COMPLETE TECHNICAL HANDOVER DOCUMENT

Version: 1.0

Status: Production Development Baseline

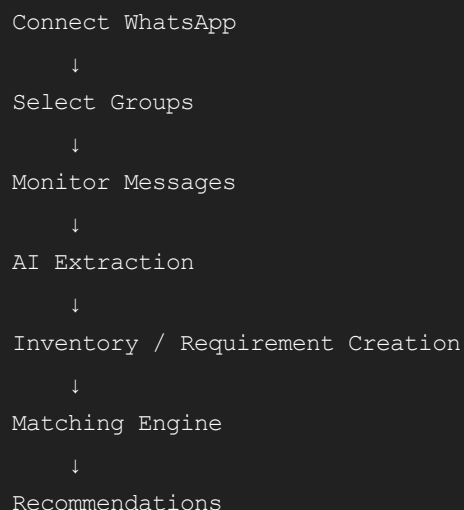
Prepared For: Future Development Team

1. PROJECT OBJECTIVE

The purpose of the system is to automatically monitor WhatsApp groups, classify messages using AI, extract structured real-estate data, store it in MongoDB, and ultimately match buyers with properties.

The platform is designed as a multi-tenant SaaS product.

Each tenant can:



```
graph TD; A[Connect WhatsApp] --> B[Select Groups]; B --> C[Monitor Messages]; C --> D[AI Extraction]; D --> E[Inventory / Requirement Creation]; E --> F[Matching Engine]; F --> G[Recommendations];
```

Connect WhatsApp

↓

Select Groups

↓

Monitor Messages

↓

AI Extraction

↓

Inventory / Requirement Creation

↓

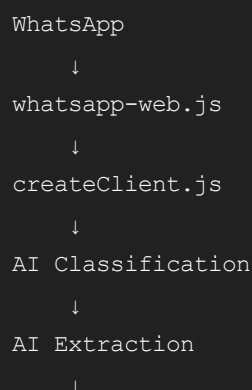
Matching Engine

↓

Recommendations



2. SYSTEM ARCHITECTURE



```
graph TD; A[WhatsApp] --> B[whatsapp-web.js]; B --> C[createClient.js]; C --> D[AI Classification]; D --> E[AI Extraction]; E --> F[...];
```

WhatsApp

↓

whatsapp-web.js

↓

createClient.js

↓

AI Classification

↓

AI Extraction

↓



MongoDB



INVENTORY
REQUIREMENT
OTHER



Future Matching Engine

3. TECHNOLOGY STACK

Backend

Node.js
Express
MongoDB
Mongoose



WhatsApp

whatsapp-web.js
puppeteer-core



AI

OpenAI API
Model: gpt-5.6-luna



Process Manager

PM2



Frontend

EJS
Bootstrap



4. PROJECT STRUCTURE

Important folders:

```
routes/  
services/  
models/  
views/  
Prompts/  
public/
```



5. MOST IMPORTANT FILES

AI Prompt

```
Prompts/whatsappExtractionPrompt.v2.txt
```



This is the primary AI behavior file.

Never hardcode prompts elsewhere.

All extraction behavior changes must happen here first.

AI Extraction Service

```
services/whatsappAIExtractor.js
```



Responsible for:

```
Loading prompt  
Loading learning examples  
Calling OpenAI  
Returning AI output
```



Main Function:

```
<> JavaScript  
  
extractWithAI()
```



Classification Service

```
services/classifyWhatsappMessage.js
```



Responsible for:

INVENTORY
REQUIREMENT
OTHER



classification.

WhatsApp Core

services/tenantWhatsapp/createClient.js



Most important operational file.

Contains:

Message listener
Inventory save logic
Requirement save logic
Recovery logic
AI integration



Any WhatsApp processing changes are done here.

Session Recovery

services/tenantWhatsapp/restoreSessions.js



Responsible for:

Server Restart
↓
Restore Sessions
↓
Create Client



Without this file tenants will not reconnect after restart.

6. MULTI-TENANT ARCHITECTURE

Each tenant has:

Tenant



WhatsApp Session



Groups



Messages



Collections:

tenants

tenantwhatsapps

whatsappgroups



7. MESSAGE FLOW

Step 1

WhatsApp receives:

1 bhk needed in Kharadi



Step 2

Message enters:

⌞ JavaScript

```
client.on('message')
```



inside:

createClient.js



Step 3

Message stored in:

whatsappmessages



Step 4

Message classified.

Possible outputs:

```
INVENTORY
REQUIREMENT
OTHER
```



Step 5

AI extraction executed.

Step 6

Mongo records created.

8. AI EXTRACTION

Current AI contract:

</> JSON



```
{
  "classification": "INVENTORY",
  "multipleRecords": true,
  "records": []
}
```

IMPORTANT:

The platform no longer uses:

</> JSON



```
{
  "propertyType": "",
  "bhk": ""
}
```

as the primary format.

Everything must use:

</> JSON



```
{
  "records": []
}
```

architecture.

9. MULTI INVENTORY EXTRACTION

STATUS

COMPLETED



Example Input:

7 rental properties in one message



AI Output:

JSON



```
{
  "multipleRecords": true,
  "records": [
    {},
    {},
    {},
    {},
    {},
    {},
    {}
  ]
}
```

Database Result:

7 whatsappinventories records



Implementation:

createClient.js



Uses:

JavaScript



```
for (...) {
  await WhatsappInventory.create(...)
}
```

10. MULTI REQUIREMENT EXTRACTION

STATUS

COMPLETED



Example:

Need:



1bhk Kharadi
2bhk Viman Nagar
PG Magarpatta

AI returns:

JSON



```
{
  "multipleRecords": true,
  "records": [
    {},
    {},
    {}
  ]
}
```

Database Result:

3 whatsapprequirements records



11. MONGODB COLLECTIONS

whatsappmessages

Stores:

Raw Message
AI Output
Classification
Token Usage



whatsappinventories

Stores:

Property Inventory



Fields:

BHK

Budget

Location

Property Type



whatsapprequirements

Stores:

Buyer Requirements



Fields:

BHK

Budget

Location

Property Type



learningcorrections

Stores AI corrections.

This is the knowledge base.

12. AI LEARNING SYSTEM

STATUS

COMPLETED



Purpose:

Allow admins to teach AI without code changes.

Workflow:

```
Other Messages
  ↓
Correct AI
  ↓
Save Correction
  ↓
learningcorrections
```



Later:

```
loadLearningExamples()
  ↓
Prompt Injection
  ↓
extractWithAI()
```



13. LEARNING CORRECTION COLLECTION

Model:

```
models/LearningCorrection.js
```



Stores:

```
Original Message
AI Output
Corrected Output
```



Example:

```
double sharing pg needed
```



AI:

```
<> JSON

{
  "propertyType": "APARTMENT"
}
```



Correction:

 JSON



```
{  
  "propertyType": "PG"  
}
```

14. LEARNING CORRECTION ROUTES

File:

routes/learningCorrectionRoutes.js



Implemented:

```
POST /add  
GET /list  
GET /create/:messageId  
GET /edit/:id  
POST /edit/:id
```



15. LEARNING CORRECTION VIEWS

Files:

```
views/learningCorrections.ejs  
views/learningCorrectionCreate.ejs  
views/learningCorrectionEdit.ejs
```



16. COPY AI OUTPUT

STATUS

COMPLETED



Workflow:

```
AI Output  
↓  
Copy AI Output
```



↓

Purpose:

Avoid retyping large JSON.

17. EDIT CORRECTION

STATUS

COMPLETED



Workflow:

Corrections List



↓

Edit

↓

Update

Purpose:

Modify learning examples.

18. WHY DELETE WAS NOT IMPLEMENTED

Delete removes knowledge.

Current architecture:

learningcorrections



↓

Prompt Learning

Deleting:

Correction



↓

Removed

↓

AI no longer sees it

Therefore Delete was intentionally postponed.

19. PROMPT VERSIONING

Current prompt:

```
Prompts/whatsappExtractionPrompt.v2.txt
```



Future versions:

```
v3
```

```
v4
```

```
v5
```



Never overwrite older versions.

20. PM2 OPERATIONS

Start:

```
</> cmd
```



```
pm2 start app.js --name myworld
```

Stop:

```
</> cmd
```



```
pm2 stop myworld
```

Restart:

```
</> cmd
```



```
pm2 restart myworld
```

Logs:

```
</> cmd
```



```
pm2 logs myworld
```

Save:

```
</> cmd
```



```
pm2 save
```

21. SERVER RESTART PROCESS

Normal shutdown:

```
</> cmd  
  
pm2 stop myworld
```



Restart:

```
</> cmd  
  
pm2 restart myworld
```



After machine reboot:

```
</> cmd  
  
pm2 resurrect
```



or

```
</> cmd  
  
pm2 start myworld
```



depending on PM2 configuration.

22. TESTING PROCEDURE

AI Testing:

```
test-ai.js
```



Must be used before production deployment.

Never modify production prompt without testing.

23. DEVELOPMENT RULES

Rule 1

Never hardcode AI prompts.

Always update:

```
Prompts/whatsappExtractionPrompt.v2.txt
```



Rule 2

Never remove:

```
</> JSON
```



```
records[]
```

architecture.

Rule 3

Never bypass:

```
learningcorrections
```



system.

Rule 4

Always test using:

```
test-ai.js
```



before deployment.

Rule 5

Any database changes must update:

```
models/
```



first.

24. KNOWN FUTURE FEATURES

Inventory Matching

Requirement



Matching Inventory



Requirement Matching

Inventory



Matching Requirements



Recommendation Engine

Inventory



Best Buyers



Auto Shortlisting

AI



Top Matches



Analytics

Hot Locations

Rental Trends

Budget Trends

Property Demand



25. CURRENT PROJECT STATUS

WhatsApp Integration	COMPLETE
AI Classification	COMPLETE
AI Extraction	COMPLETE
Prompt Learning	COMPLETE
Correction Workflow	COMPLETE
Copy AI Output	COMPLETE
Edit Correction	COMPLETE
Multi Inventory	COMPLETE
Multi Requirement	COMPLETE
Recovery Processing	COMPLETE
Live Processing	COMPLETE
PM2 Deployment	COMPLETE



NEXT MAJOR DEVELOPMENT PHASE

Inventory ↔ Requirement Matching Engine



This should be considered the official baseline from which future development continues.